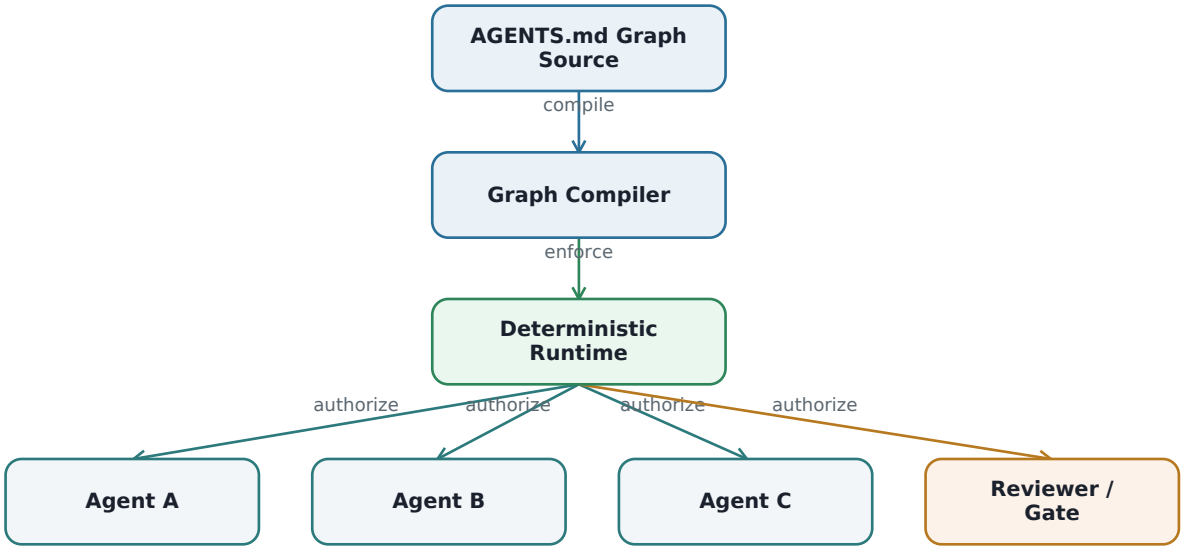
# CONSTRAINED ORCHESTRATION GRAPH

A deterministic governance architecture for multi-agent execution

## Technical Architecture Paper

**Core rule: Models may propose transitions. The orchestration runtime decides whether those transitions are legal.**




Scope: declarative policy, graph compilation, state-machine enforcement, multi-agent scheduling, authority separation, evidence receipts, invalidation, atomic runtime state, recovery, and execution-adapter boundaries.

Version 1.0 - October 2026

# Abstract

This paper specifies a multi-agent orchestration architecture in which natural-language agents are participants inside a deterministic execution graph rather than final authorities over workflow progression. The source policy is expressed as a typed graph embedded in repository instructions, while live state, evidence, identity, and receipts are held separately by a runtime designed to fail closed. The model can reason, delegate, implement, review, and request advancement; the runtime decides whether the requested transition is legal.

The design separates four concerns that are frequently collapsed in agent systems: execution dependency, communication, authority, and evidence. This separation permits parallel work without overlapping write authority, prohibits self-review, invalidates stale approvals after artifact mutation, and makes integration dependent on verified graph conditions rather than model assertion. The result is a governance-oriented control plane for multi-agent work that can sit above Codex or another agent execution adapter without pretending that a Markdown file itself is a scheduler or message bus.

> **The design target is not merely a graph-shaped prompt. It is an enforceable orchestration system.**

# Contents

1. Problem Statement and Design Goal
2. System Boundary and Terminology
3. Formal Graph Model
4. Declarative Graph Source in AGENTS.md
5. Graph Discovery, Merge, and Compilation
6. Runtime State Machine
7. Ready-Set Evaluation and Scheduling
8. Parallelism and Write-Scope Enforcement
9. Authority Graph and Separation of Duties
10. Agent Identity and Claims
11. Evidence Receipts and Verification
12. Invalidation and Stale Authority
13. Runtime Persistence and Event Ledger
14. Atomicity, Concurrency, and Recovery
15. Execution Adapter Boundary
16. Failure Semantics
17. Security and Governance Invariants
18. Reference Algorithms
19. End-to-End Execution Walkthrough
20. Verification and Adversarial Test Plan
21. Suggested Implementation Layout
22. Extensions and Future Work

Conclusion

Appendix A. Reference Schema

Appendix B. Example Runtime State

Appendix C. Example Receipts

# 1. Problem Statement and Design Goal

Agent systems often begin with procedural instructions: inspect the repository, delegate work, run tests, ask another agent to review, and integrate when ready. This is useful guidance, but guidance remains model-interpreted. If the model itself decides whether prerequisites are satisfied, then the same component that performs the work also controls the definition of completion. That collapses execution and authority into one reasoning loop.

The design goal of the constrained orchestration graph is to separate those responsibilities. Agents perform cognitive and technical work. A deterministic runtime enforces the legal progression of that work. The runtime does not replace agent reasoning; it constrains when reasoning may produce state-changing effects.

Primary objective. A model may request that a node advance, but no model - including the parent orchestrator - can force the graph through an illegal transition by asserting that the transition should occur.

## 1.1 Design principles

• Fail closed when graph semantics, evidence, identity, scope, or runtime state are ambiguous.

• Separate static policy from live execution state.

• Separate execution dependency from communication and authority.

• Bind approvals to exact artifacts or artifact hashes.

• Use actual filesystem and repository evidence rather than agent self-report when verifying changed scope.

• Make concurrent writes legal only when non-overlap can be established.

• Keep execution-adapter concerns separate from graph authority.

• Preserve deterministic, inspectable state transitions and event history.

# 2. System Boundary and Terminology

| Term | Meaning |
|---|---|
| Graph source | The declarative policy representation, typically embedded in repository instructions. |
| Compiler | Discovers applicable graph fragments, validates them, merges them, and emits an effective execution graph. |
| Runtime | Owns legal transitions, READY evaluation, claims, evidence validation, and persisted run state. |
| Parent agent | The orchestration-facing model that asks what is legal, spawns work through an adapter, and submits results. |
| Execution adapter | The mechanism that actually creates or communicates with agents. Codex multi-agent tools are one possible |
| Node | An executable agent job, gate, review, integration step, or other graph operation. |
| Edge | A typed relation between nodes, such as depends_on, requires_review, authorizes, or gates. |
| Receipt | Structured evidence describing what actually happened during node execution. |
| Claim | An atomic binding between a READY node and a concrete executor identity. |

**Figure 1. System boundary**

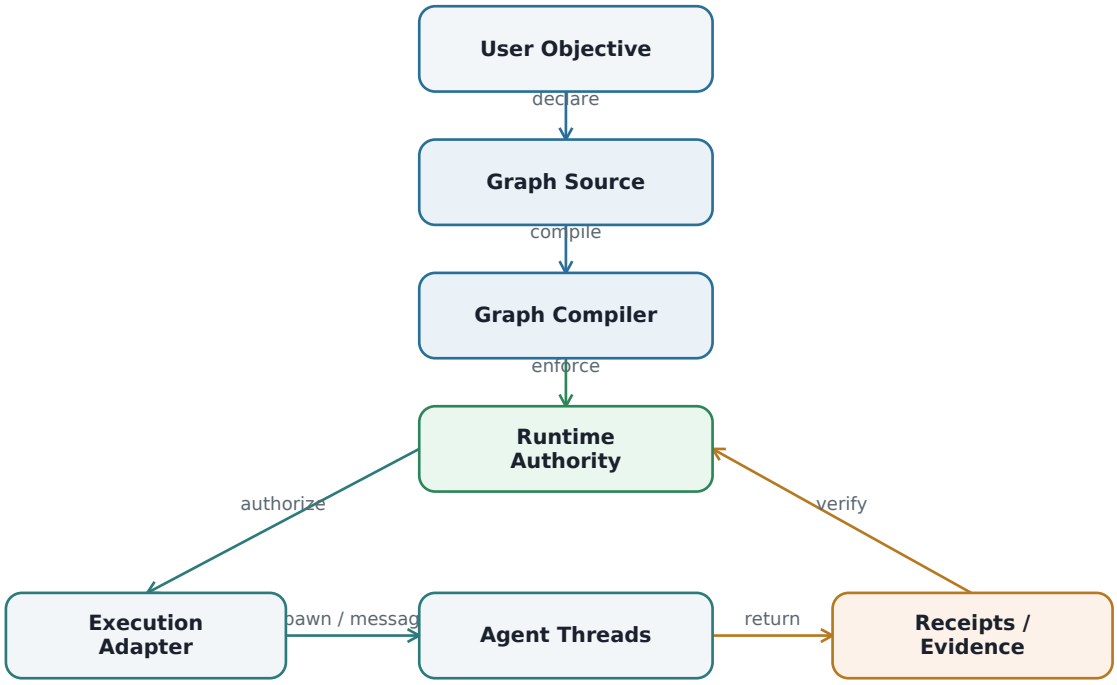



Figure 1 - The graph runtime authorizes progression; the execution adapter performs work but does not own graph legality.

# 3. Formal Graph Model

Let the orchestration graph be G = (V, E), where V is the set of nodes and E is the set of typed directed edges. Each node has a stable identifier, kind, role, access mode, optional write scope, and runtime state. Each edge has a semantic type whose rules determine scheduling, authority, or information flow.

```
G = (V, E)

V = {agent jobs, reviewers, validators, gates, integrators, ...}
E = {(u, v, type)}

READY(v) = true only if every hard predecessor requirement for v is satisfied.
```

## 3.1 Three distinct graph dimensions

A robust agent system should not collapse all relationships into a single generic dependency. The execution graph, communication graph, and authority graph answer different questions.

| Graph dimension | Question answered | Example |
|---|---|---|
| Execution | What must complete before something else may run? | planner -> worker |
| Communication | Who must inform or report to whom? | worker -> parent |
| Authority | Who can review, authorize, or gate whose work? | worker -> reviewer -> validator |

The graph compiler may store these dimensions in one typed edge set, but the runtime must preserve their semantic distinction.

## 3.2 Edge type semantics

| Edge type | Hard scheduling dependency? | Semantic effect |
|---|---|---|
| depends_on | Yes | Target cannot become READY until source satisfies success condition. |
| delegates_to | Usually yes | Source authorizes creation or assignment of target work. |
| requires_review | Yes | Target review must occur after source work and must use distinct identity. |
| authorizes | Yes for guarded target | Target may advance only after explicit authorization condition is satisfied. |
| gates | Yes | Target remains blocked until gate succeeds. |
| informs | No | Creates communication obligation but does not alone block scheduling. |

Unknown edge types are invalid. They must not silently degrade into generic dependencies because doing so changes authority semantics without explicit approval.

# 4. Declarative Graph Source in AGENTS.md

The repository instruction file remains human-readable Markdown, but it contains one canonical machine-readable graph block. The file therefore serves two audiences: humans reviewing the operating constitution and the compiler constructing the effective orchestration graph.

````
```yaml agents-graph
version: 1
graph:
  id: repository_change
  policy:
    fail_closed: true
    max_parallel: 4
    require_receipts: true
    prohibit_self_review: true
    prohibit_scope_overlap: true

  nodes:
    discover:
      kind: agent
      role: explorer
      access: read_only

    backend:
      kind: agent
      role: worker
      access: write_scoped
      write_scope: ["backend/**"]

    review:
      kind: agent
      role: reviewer
      access: read_only

    validate:
      kind: gate

    integrate:
      kind: integration
````
````

```
  edges:
    - {from: backend, to: review, type: requires_review}
    - {from: review, to: validate, type: authorizes}
    - {from: validate, to: integrate, type: gates}
```

The graph source specifies what may and must happen. It does not store volatile facts such as current agent progress, current thread IDs, or transient status. Those belong to runtime state.

> **Policy is source. State is runtime. Evidence is history.**

# 5. Graph Discovery, Merge, and Compilation

Repository-scoped instruction files naturally permit graph composition. A root graph can define universal controls; deeper graph fragments can specialize backend, frontend, security, or other subtrees. Override files may replace same-level policy under explicit semantics.

```
repo/
  AGENTS.md                 # root graph
  backend/
    AGENTS.md               # backend fragment
  frontend/
    AGENTS.md               # frontend fragment
  security/
    AGENTS.override.md      # explicit security override
```

## 5.1 Compilation pipeline

- Discover applicable instruction files from project root toward current working scope.
- Extract canonical graph blocks and associate each block with its directory scope.
- Parse against a versioned schema.
- Apply root-to-leaf composition rules.
- Require explicit override markers for duplicate node IDs or changed semantics.
- Resolve references, role names, scopes, and edge endpoints.
- Build the hard-dependency subgraph and detect cycles.
- Validate authority relationships and prohibited combinations.
- Compute a canonical serialization and graph fingerprint.
- Emit the immutable compiled graph snapshot used by the run.

## 5.2 Fail-closed compile errors

| Condition | Required behavior |
|---|---|
| Unknown node reference | Compilation fails. |
| Duplicate node ID without explicit override | Compilation fails. |
| Unknown edge type | Compilation fails. |
| Cycle in v1 hard-dependency graph | Compilation fails. |
| Ambiguous override | Compilation fails. |
| Invalid write scope | Compilation fails. |
| Conflicting authority relationships | Compilation fails. |

# 6. Runtime State Machine

Every executable node is governed by an explicit runtime state machine. State is not a free-form field that the model may assign. Each transition has a validation predicate and produces an event.

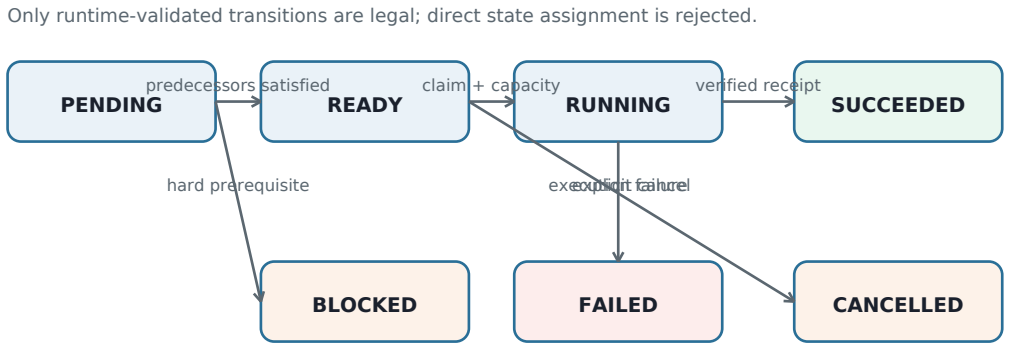



Figure 2 - Core node state machine. Additional implementation-specific recovery states may be added if they preserve deterministic semantics.

## 6.1 Transition examples

| Transition | Minimum legal condition |
|---|---|
| PENDING -> READY | Every hard predecessor and authority prerequisite required for scheduling is satisfied. |
| READY -> RUNNING | Atomic claim succeeds; capacity exists; executor identity is valid; scope conflicts are absent. |
| RUNNING -> SUCCEEDED | Receipt verifies; required outputs exist; scope is respected; validation conditions pass. |
| RUNNING -> FAILED | Execution reports failure or verification rejects required evidence. |
| PENDING -> BLOCKED | A required dependency is known to be unavailable or failed under policy. |
| READY -> CANCELLED | Explicit authorized cancellation. |

# 7. Ready-Set Evaluation and Scheduling

The scheduler does not ask the model which nodes feel ready. It computes the READY set from the compiled graph and current runtime state. A node is READY when every hard edge into that node is satisfied according to that edge type, and when no policy-level blocker prevents execution.

```
ready = []
for node in graph.nodes:
    if state[node] != PENDING:
        continue
    if all(hard_requirement_satisfied(edge) for edge in incoming(node)):
        if policy_allows_scheduling(node):
            ready.append(node)
```

The runtime should return both READY and BLOCKED explanations so the parent can understand the legal frontier without inventing its own reasoning about graph state.

```
READY NOW:
  - backend_worker
  - frontend_worker

BLOCKED:
  - review: waiting on backend_worker
  - validate: review authorization missing
  - integrate: validation gate unsatisfied
```

# 8. Parallelism and Write-Scope Enforcement

Parallel execution is legal only when graph dependencies permit it and the runtime can establish that write-capable nodes have non-overlapping effective scopes. Read-only nodes may run concurrently when other constraints allow.

```
parallel_legal(A, B) =
    A is READY
    and B is READY
    and running_count < max_parallel
    and (read_only(A) or read_only(B) or disjoint(write_scope(A), write_scope(B)))
```

Glob/path overlap must be evaluated conservatively. If the runtime cannot prove non-overlap, it serializes the work. The final scope check is based on actual changed files or repository diffs, not on what the agent says it changed.

## 8.1 Example

| Worker | Authorized scope | Observed changes | Result |
|---|---|---|---|
| backend_worker | backend/** | backend/api.py, backend/auth.py | Accept |
| frontend_worker | frontend/** | frontend/App.tsx | Accept |
| backend_worker | backend/** | backend/api.py, docs/readme.md | Reject: scope violation |

# 9. Authority Graph and Separation of Duties

Completion of work is not equivalent to authority to integrate that work. The graph therefore separates production, review, validation, and integration. A worker may produce an artifact; a distinct reviewer may approve a specific version; a validator may establish test evidence; an integrator may act only after those conditions are satisfied.

```
IMPLEMENTED != AUTHORIZED != VALIDATED != INTEGRATED
```

**Figure 3. Authority chain**

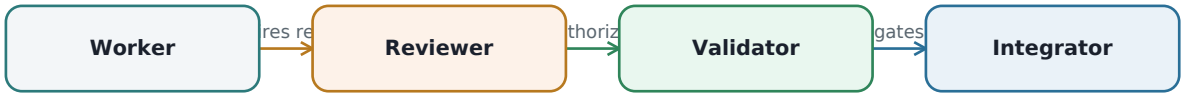



Figure 3 - Each stage has a distinct authority meaning. A worker cannot self-certify a required review.

# 10. Agent Identity and Claims

Each RUNNING node must be bound to a concrete execution identity. This identity may be a Codex thread identifier, process identity, worker token, or another adapter-specific identifier, but it must be stable for the duration of the claim.

```
graph node  --->  execution identity

backend_worker ---> agent-thread-7f92
review         ---> agent-thread-a183
```

Claims are acquired atomically. Two orchestrators racing to claim the same node cannot both succeed. Identity is also required for no-self-review rules, receipt ownership, responsibility attribution, and restart recovery.

# 11. Evidence Receipts and Verification

A receipt is structured evidence about node execution. It is not authority by itself. The runtime verifies that the receipt is internally consistent, belongs to the correct run and node, matches the claimed executor, reflects the allowed scope, and satisfies the graph conditions required for success.

| Receipt field | Purpose |
|---|---|
| run_id | Binds evidence to one orchestration run. |
| node_id | Identifies the graph operation. |
| executor_id | Binds result to the claimed execution identity. |
| started_at / finished_at | Provides execution interval. |
| input_hash | Binds work to the exact input/reference state. |

| Receipt field | Purpose |
|---|---|
| changed_files | Supports write-scope verification. |
| commands | Records material execution steps. |
| validation_results | Records test/gate evidence. |
| exit_status | Captures execution outcome. |
| artifact_hash / diff_hash | Binds review and downstream authority to exact output. |

Where possible, repository state should be independently recomputed. For example, changed_files should be verified from Git diff or equivalent repository evidence rather than accepted solely from the receipt payload.

# 12. Invalidation and Stale Authority

Authority is valid only for the artifact that was reviewed or validated. If the underlying artifact changes, downstream approvals that depended on its previous fingerprint must be invalidated.

```
implementation hash = X
review approves X

implementation changes -> hash = Y

review(X) cannot authorize Y
```

Invalidation propagates through dependency and authority edges. The runtime should mark affected downstream nodes invalid, blocked, or pending according to a documented deterministic policy. It must never silently retain approval for content that no longer matches the approved artifact.

# 13. Runtime Persistence and Event Ledger

Live execution state is stored outside the graph source. One possible repository-local runtime layout is shown below. Exact paths may follow project conventions.

```
.agents/
  runtime/
    <run-id>/
      graph.snapshot.json
      state.json
      claims.json
      events.jsonl
      receipts/
      results/
```

## 13.1 Event ledger

Important state changes are appended to an ordered event ledger. The ledger supports post-run reconstruction, debugging, and auditability.

| Field | Description |
|---|---|
| run_id | Owning run. |
| sequence | Monotonic deterministic event sequence. |

| Field | Description |
|---|---|
| timestamp | Recorded event time. |
| node_id | Affected node. |
| executor_id | Relevant execution identity, if any. |
| previous_state | State before event. |
| new_state | State after event. |
| event_type | Claim, transition, receipt, invalidation, recovery, etc. |
| reason | Machine-readable reason code plus optional detail. |
| receipt_ref | Evidence reference when applicable. |

# 14. Atomicity, Concurrency, and Recovery

Runtime state mutation must be atomic. A claim operation is a compare-and-set transaction: the node must still be READY, unclaimed, and policy-valid at the instant the claim commits. File locks, database transactions, or equivalent repository-native primitives may be used.

## 14.1 Crash recovery

- Persist the compiled graph snapshot and fingerprint before executing any node.
- Persist claims before work is launched.
- On restart, identify RUNNING nodes whose executor is no longer reachable.
- Do not automatically assume success from partial artifacts.
- Reconcile repository evidence and receipts before deciding whether to retry, fail, or recover.
- Use idempotent state transitions where practical.
- Never leave a half-written state file as the authoritative record.

# 15. Execution Adapter Boundary

The orchestration runtime and the agent execution mechanism are deliberately separate. The runtime answers what may happen. The adapter performs authorized work. This keeps the architecture valid whether the adapter is Codex multi-agent tools, another agent framework, isolated worker processes, or a test harness.

```
Runtime:   "backend_worker is READY and may be claimed."
Parent:    claims backend_worker
Adapter:   spawns concrete worker identity
Worker:    executes bounded task
Adapter:   returns result
Parent:    submits receipt
Runtime:   verifies evidence and either accepts or denies transition
```

The repository must not pretend to import or control private execution internals that are unavailable. If the execution mechanism is available only to the parent agent, the parent acts as an adapter client

while the runtime remains the authority over legal graph progression.

# 16. Failure Semantics

| Failure condition | Required behavior |
|---|---|
| Worker failure | Mark FAILED; block dependents unless retry policy explicitly permits recovery. |
| Reviewer rejection | Record rejection evidence; keep guarded downstream nodes blocked. |
| Validation failure | Gate fails; integration remains illegal. |
| Executor crash | Reconcile claim and evidence; do not assume success. |
| Malformed receipt | Reject completion. |
| Scope violation | Reject completion and flag governance violation. |
| Graph mutation during run | Keep run pinned to snapshot or require explicit migration; never silently switch. |
| Stale receipt | Reject if input/artifact fingerprint does not match current expected state. |
| Duplicate claim | Exactly one claim succeeds. |
| Illegal transition attempt | Return deterministic denial and reason code. |

# 17. Security and Governance Invariants

The following invariants define the minimum governance contract. An implementation that cannot enforce them should not claim to implement the constrained orchestration graph.

I1 - No illegal transition: every state change is validated against graph semantics and current state.

I2 - No unbound execution: every RUNNING agent node has a concrete claimed executor identity.

I3 - No double claim: a node has at most one active executor claim.

I4 - No self-review when independence is required: producer identity cannot satisfy required reviewer identity.

I5 - No stale authority: approval is bound to an exact artifact/input fingerprint.

I6 - No unauthorized writes: successful completion requires observed changes to remain inside authorized scope.

I7 - No silent graph mutation: every run is bound to a compiled graph fingerprint.

I8 - No evidence-free success when receipts are required.

I9 - No parent override: the parent agent cannot directly mutate protected runtime state to bypass denial.

I10 - No ambiguity promotion: unresolved graph or scope ambiguity blocks rather than authorizes execution.

# 18. Reference Algorithms

## 18.1 Compile graph

```
function compile(scope):
    files = discover_instruction_files(scope)
    fragments = []
    for file in files:
        block = extract_graph_block(file)
        if block:
            fragments.append(parse_and_validate(block))

    effective = merge_root_to_leaf(fragments)
    resolve_references(effective)
    validate_edge_types(effective)
    validate_authority(effective)
    reject_cycles(hard_dependency_subgraph(effective))
    fingerprint = hash(canonical_serialize(effective))
    return CompiledGraph(effective, fingerprint)
```

## 18.2 Claim node

```
function claim(node, executor):
    begin atomic transaction
    assert state[node] == READY
    assert claim[node] == NONE
    assert executor is valid
    assert capacity_available()
    assert no_write_scope_conflict(node)
    claim[node] = executor
    state[node] = RUNNING
    append_event(CLAIMED)
    commit
```

## 18.3 Complete node

```
function complete(node, receipt):
    assert state[node] == RUNNING
    verify_receipt_identity(node, receipt)
    verify_input_fingerprint(node, receipt)
    verify_outputs(node, receipt)
    verify_changed_scope(node, receipt)
    verify_required_validations(node, receipt)
    invalidate_stale_downstream_authority_if_needed(node, receipt)
    state[node] = SUCCEEDED
    append_event(COMPLETED)
    recompute_ready_set()
```

# 19. End-to-End Execution Walkthrough

Consider the following graph: discover -> plan -> {worker_a, worker_b} -> review -> validate -> integrate. The workers have disjoint write scopes. The reviewer is read-only and must be a distinct identity from either worker.

**Figure 4. Reference execution DAG**

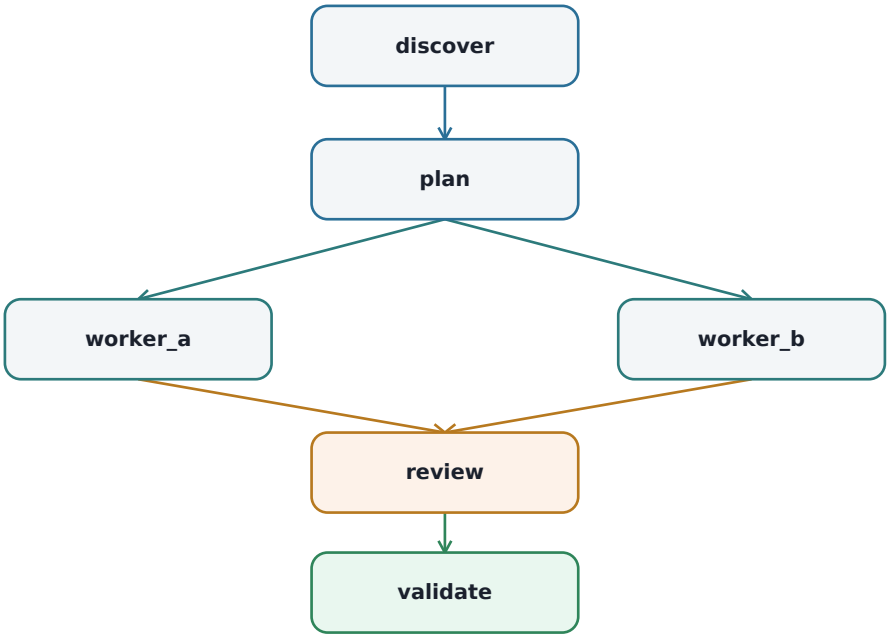



Figure 4 - worker_a and worker_b may run concurrently because they are independent and write scopes are disjoint.

| Step | Event | Runtime consequence |
|---|---|---|
| 1 | Run initialized | Compiler emits graph snapshot and fingerprint. discover becomes READY. |
| 2 | Discover claimed | Parent asks runtime, claims discover, spawns explorer through adapter. |
| 3 | Discover completes | Receipt accepted. plan becomes READY. |
| 4 | Plan completes | worker_a and worker_b become READY simultaneously. |
| 5 | Parallel workers | Runtime verifies non-overlapping write scopes and admits both claims. |
| 6 | Workers return | Receipts are checked against actual repository diffs and input hashes. |
| 7 | Review admitted | Only after both workers SUCCEED does review become READY. |
| 8 | No self-review | Runtime rejects any reviewer identity equal to a producer identity. |
| 9 | Validation gate | Successful review authorizes validate; test evidence must pass. |
| 10 | Integration | Only after validate succeeds does integrate become legal. |
| 11 | Mutation check | Any artifact mutation after review invalidates that review and re-blocks integration. |
| 12 | Terminal success | Run completes only when every required terminal condition is satisfied. |

# 20. Verification and Adversarial Test Plan

The test strategy must prove denial behavior as aggressively as happy-path behavior. The implementation is not complete merely because a legal run succeeds. It must also prove that illegal progression is consistently rejected.

1. Valid graph compilation

2. Invalid schema rejection
3. Unknown node reference rejection
4. Duplicate node rejection
5. Invalid override rejection
6. Cycle detection
7. Nested fragment composition
8. Override behavior
9. READY-set evaluation
10. Legal state transitions
11. Illegal state transitions
12. Duplicate claim race
13. Parallel read-only execution
14. Parallel disjoint writers
15. Overlapping writer rejection
16. Out-of-scope write rejection
17. Self-review rejection
18. Missing authorization
19. Missing gate
20. Validation failure
21. Malformed receipt
22. Stale receipt
23. Artifact mutation after review
24. Downstream invalidation
25. Crash/restart recovery
26. Graph fingerprint mismatch
27. End-to-end success
28. Manual jump-to-integration denial

### 20.1 Hostile verification standard

At minimum, attempt to: claim the same node from two processes, mark integration successful before validation, use a worker identity as its own reviewer, submit a receipt for the wrong graph fingerprint, alter a reviewed artifact, write outside authorized scope, and manually mutate persisted state. Each attempt should either be technically prevented or detected and denied before authority advances.

## 21. Suggested Implementation Layout

The implementation should remain small, composable, and testable without launching an LLM. Pure graph logic should be isolated from I/O and execution-adapter behavior.

```
orchestration/
  schema.*
  discovery.*
```

```
compiler.*
graph_validation.*
state_machine.*
ready_set.*
claims.*
scope_verifier.*
receipts.*
invalidation.*
persistence.*
events.*
cli.*
adapters/
  codex.*
  test.*
tests/
```

The exact language and file layout should follow the repository being integrated. The architectural requirement is separation of responsibilities, not a particular framework.

# 22. Extensions and Future Work

- Conditional edges and policy predicates beyond simple DAG semantics.
- Explicit retry budgets and retry-specific authority rules.
- Quorum review, multi-signature authorization, and threshold gates.
- Capability tokens for bounded execution authority.
- Cross-repository graph composition with signed graph fragments.
- Policy version migration for long-running workflows.
- Artifact-level provenance graphs and cryptographic receipt chains.
- Formal verification of transition invariants.
- Resource-aware scheduling and hardware admission control.
- Human approval nodes with explicit legal authority boundaries.

# Conclusion

The constrained orchestration graph changes the role of the model from workflow sovereign to governed participant. Agents remain responsible for reasoning, exploration, implementation, review, and communication, but they cannot declare their own prerequisites satisfied merely through language. A deterministic runtime evaluates typed graph semantics, verifies evidence, binds authority to exact artifacts, controls concurrency, and decides whether the graph may advance.

The architecture therefore preserves the flexibility of agent reasoning while placing workflow progression behind explicit, inspectable, testable constraints. The graph source states what is allowed. Runtime state records what is happening. Receipts record what happened. The transition engine decides what may happen next.

> **The model can ask. The graph decides.**

# Appendix A. Reference Schema

```
version: 1
graph:
  id: string
  policy:
    fail_closed: boolean
    max_parallel: integer
    require_receipts: boolean
    prohibit_self_review: boolean
    prohibit_scope_overlap: boolean

  nodes:
    <node_id>:
      kind: agent | gate | integration | human
      role: string | null
      access: read_only | write_scoped | privileged
      write_scope: [glob, ...]
      required_outputs: [string, ...]
      retry_policy: optional

  edges:
    - from: node_id
      to: node_id
      type: depends_on | delegates_to | requires_review | authorizes | gates | informs
      conditions: optional
```

# Appendix B. Example Runtime State

```
{
  "run_id": "run-20261005-001",
  "graph_fingerprint": "sha256:...",
  "status": "RUNNING",
  "nodes": {
    "discover": {"state": "SUCCEEDED"},
    "plan": {"state": "SUCCEEDED"},
    "worker_a": {"state": "RUNNING", "executor_id": "agent-7f92"},
    "worker_b": {"state": "SUCCEEDED", "executor_id": "agent-9a10"},
    "review": {"state": "BLOCKED"},
    "validate": {"state": "BLOCKED"},
    "integrate": {"state": "BLOCKED"}
  }
}
```

# Appendix C. Example Receipt

```
{
  "run_id": "run-20261005-001",
  "node_id": "worker_b",
  "executor_id": "agent-9a10",
  "started_at": "2026-10-05T21:10:00Z",
  "finished_at": "2026-10-05T21:16:41Z",
  "input_hash": "sha256:...",
  "artifact_hash": "sha256:...",
  "changed_files": ["frontend/App.tsx"],
  "commands": ["npm test -- --runInBand"],
  "validation_results": [{"name": "frontend-tests", "status": "passed"}],
  "exit_status": 0
}
```

A valid receipt can establish evidence that a node performed work. It becomes graph authority only after the runtime verifies that the evidence satisfies the exact requirements associated with the node and its outgoing authority edges.